

Diffusion-MoE Phase 1 Geometry Investigation

Findings report — kill-switch validity, root causes, and open questions

Prepared for review · *diffusion-moe project*

Executive Summary

The Phase 1 "kill switch" (`scripts/extract_geometry.py`) originally reported **mean $r^* = 4.2$ vs. $d_{\text{model}} = 4096$** on a real Mistral-7B model and would have printed a "manifold hypothesis holds — proceed to training" message, greenlighting the ~\$24,000 Phase 2 training spend.

That result should **not** have been trusted as-is. Investigation found:

1. **The kill-switch threshold itself was buggy** — it compared r^* against `$d_{\text{model}} \times 0.5$` , not the " d_{model} " claim it printed. Fixed.
2. **At the kill-switch's own single default bandwidth, 4 of 32 layers (1, 2, 4, 31) show a kernel-graph disconnection artifact** that trivially deflates r^* to ~1–2 regardless of the real geometry, root-caused to outlier-norm ("attention-sink"-style) tokens. A full 32-layer multiscale bandwidth sweep (§2.4) refines this: layers **17, 20, and 31** never connect under raw Euclidean distance at *any* of 9 bandwidths spanning a 256× range — bandwidth choice cannot fix them, only a different metric (cosine) does. Layers 1, 2, and 4, by contrast, **do** connect at other bandwidths in that same sweep — their disconnection is specific to the kill-switch's one default bandwidth choice, not the metric itself.
3. Even the non-disconnected layers show r^* in the range **1–11 (median 4–4.5, mean 4.6, confirmed on the full 1000-sequence sample)** — smaller than the guide's own "expected 10–50." The full multiscale sweep adds a genuine nuance here, not a resolution: across the 29 layers where both metrics connect, Euclidean and cosine agree exactly in 10, Euclidean reports a *higher* r^* in 13, and cosine reports higher in 6 — the "raw magnitude carries real structure" pattern first seen at layer 15 is the most common single direction of disagreement, but far from universal.
4. A **separate, independent bug** was found and fixed in the dataset registry (`wikitext` pointed at a deprecated, now-broken Hub repo id) that would also have broken Phase 2's evaluation step.
5. A deeper question was raised during the investigation: **low r^* justifies cheap routing, but does not by itself justify narrower per-expert FFNs** (the actual compute-savings claim, G2). This was tested with an oracle-ceiling diagnostic, first on a 5-layer sample and then across the full model (§3.3). The full-model result is more optimistic about the *geometry* and more pointed about the *router*: **23 of 32 layers show a real, currently-unexploited specialization ceiling; only 3 (layers 10, 29, 30) show the diffusion router recovering any meaningful part of it; only 6 layers show no achievable specialization by any partition at all.** Width reduction looks broadly achievable at the geometry level — the routing signal, not the underlying structure, is the bottleneck almost everywhere it was tested.
6. A follow-up token-identity diagnostic found the disconnection is **not** driven by sequence position (the classic "first-token sink" story) at all — it's driven by a specific token, the **newline character**, wherever it occurs. A broader, more diffuse punctuation/delimiter effect also appears at layer 31 specifically, though it needs a dataset-specific caveat (§4).
7. Testing whether *training* closes the gap at layers 2/4 (§3.4) — the one question training-free diagnostics structurally can't answer — required actually training something, and doing so found **two real bugs in shared, pre-existing project routing code**: `centroid_separation_loss`'s gradient has no upper bound on centroid growth (fixed with a new `clip_norm` safeguard), and, more seriously, **the router wasn't differentiating between experts at all** on real data — real

diffusion coordinates measured roughly $1e-7$ in magnitude, far too small for the existing `tau=0.1` to produce anything but a numerically-uniform softmax. Both fixes apply to the production `DiffusionMoELayer`, not just the pilot script — Phase 2 would have hit both, likely silently. **With both fixed, the corrected run showed real but modest learning (task_loss gap to the dense baseline closed roughly 42%, vs. 78% under the bugged/degenerate-dispatch version) and a new, genuine pathology: the router's load-balance oscillates between collapsing onto one expert and reasonable balance across training** — real specialization is learnable here, but the default hyperparameters don't yet train it stably. Follow-up work (§3.5) substantially resolved this: a larger batch size plus correctly-calibrated noisy top-k gating eliminated full collapse and broadened expert utilization from 2 experts to 5, after tracing the root cause to k-means++ initialization luck rather than a structural property of the layer.

Bottom line: do not proceed to Phase 2 training on the strength of the original run. The disconnection issue is now understood and partially mitigated; the width-reduction question has a real, layer-dependent answer at the geometry level — and actually training something surfaced two routing bugs serious enough that Phase 2 should not proceed until they're fixed in the production code path too (confirmed fixed here, but only exercised via the pilot so far).

1. Background

Phase 1 exists as a cheap gate before committing to expensive training: if token representations don't lie on a low-dimensional manifold ($r^* \ll d_{\text{model}}$), the diffusion-routing thesis has no structure to exploit, and the project should stop and revisit the kernel design rather than spend on training. The script loads a pretrained Mistral-7B, captures post-attention activations, fits a Nyström-approximated diffusion map per layer, and estimates the intrinsic dimension r^* via eigenvalue energy retention.

2. What Went Wrong, and How It Was Found

2.1 Bug: kill-switch threshold didn't match its own claim

`scripts/extract_geometry.py` printed " $r^* \ll d_{\text{model}}$: manifold hypothesis holds" but the actual check was:

```
if mean_r_star < d_model * 0.5: # holds for r* up to 2048/4096 – not "<" by any reading
```

Fix: tightened to an explicit order-of-magnitude check (`d_model / 10`), so a weakly-compressed result can no longer slip through as a false "holds."

2.2 Discovery: the reported r^* was likely an artifact, not a signal

The original run's `top_eigenvalues` — the largest **non-trivial** diffusion eigenvalue, after the trivial $\lambda \approx 1$ stationary eigenvalue is already dropped — sat at or above roughly 0.95 on several layers, hitting **exactly 1.0** on layer 31. That is the signature of a Markov chain with more than one eigenvalue at 1: the kernel graph has fragmented into near-disconnected components. When that happens, the energy-retention formula for r^* collapses to roughly 1–2 regardless of the real underlying geometry — which is almost certainly why every layer originally came back tiny instead of the guide's expected 10–50.

Two candidate root causes were identified:

- **Hypothesis A — degenerate bandwidth:** `bandwidth_median_heuristic` falls back to `~2.2e-16` (machine epsilon) if the median pairwise landmark distance is exactly zero — which duplicate/near-duplicate landmark points (e.g. from bf16-rounding collapse) could trigger.
- **Hypothesis B — genuine outlier-norm tokens:** Llama/Mistral-family models are documented in the literature to have a handful of extreme-norm "attention-sink" / "massive-activation" tokens (e.g. the BOS position) whose hidden-state norm is orders of magnitude larger than normal tokens — the median-heuristic bandwidth (tuned to the bulk) gives near-zero kernel affinity to them, isolating them as their own component.

Diagnostics added (`scripts/extract_geometry.py` , `pool_diagnostics`): per-layer `eps` , `duplicate_fraction` , and `token_norm_max_to_median` , plus a `likely_disconnected` flag (fires when the leading non-trivial eigenvalue exceeds 0.999).

2.3 Rerun with diagnostics: Hypothesis B confirmed, Hypothesis A ruled out

Layer	r*	Disconnected	eps	norm max/median	duplicate frac
1	3	YES	0.0196	5.1	0.005
2	8	YES	0.0368	5.6	0.005
4	11	YES	0.0626	4.0	0.005
31	2	YES	45.4	24.6	0.005

(all other 28 layers: not disconnected, r* range 1–11, median 4, duplicate_fraction flat at 0.005 throughout)

- **Hypothesis A ruled out:** the minimum `eps` across all 32 layers was 0.00913 — nowhere near machine epsilon — and `duplicate_fraction` is a flat, layer-independent 0.5% everywhere, not concentrated on the flagged layers. The zero-median fallback never fired.
- **Hypothesis B supported:** `token_norm_max_to_median` spikes specifically on the flagged layers (4–24.6×), concentrated in the earliest layers (1, 2, 4 — where sink behavior is known to get established) and violently at the very last layer (31, 24.6× — consistent with known massive-activation blowup right before the LM head). This pattern matches the attention-sink literature closely.

Confirmed on the full 1000-sequence sample: a complete rerun of this exact diagnostic (fixed threshold, all 32 layers, the original wikipedia-sourced 1000-sequence pool) reproduced the same four disconnected layers and essentially identical r* statistics (mean 4.6, median 4.5, range 1–11) — the 5-layer/200-sequence figures above were not a sampling fluke.

2.4 Multiscale bandwidth sweep: direct confirmation on real data, now at full 32-layer coverage

Motivated by a parallel research idea (multiscale diffusion maps, characterizing scale behavior via a dyadic sigma ladder rather than a single point estimate — directly analogous to how a Swiss roll only "unfolds" in a bounded window of scales), a reusable sweep was built (`geometry/multiscale.py`) and run on the real model's activations, in both raw-Euclidean and cosine/angular metrics, across a 9-point dyadic bandwidth ladder spanning a 256× range. First run on 5 layers (the 4 flagged plus a clean control, layer 15); since extended to **all 32 layers** (`results/multiscale_full32/`).

Which layers does bandwidth choice alone not save? Only **17, 20, and 31** never connect under raw Euclidean distance at *any* of the 9 bandwidths tested — this is a stronger, metric-level disconnection than the kill switch's single-bandwidth flag catches, and two of these three (17, 20) were **not** among the kill

switch's original 4 flagged layers at all. Cosine cleanly resolves all three. Layers 1, 2, and 4 — the other three the kill switch flagged — **do** connect under Euclidean at other bandwidths in this same sweep; their disconnection is specific to the kill switch's one default (median-heuristic) bandwidth choice, not an intrinsic property of the metric. That's a real, useful distinction: for layers 1/2/4 a bandwidth-choice fix might suffice; for 17/20/31, only a metric change does.

Where both metrics connect, do they agree? Across the 29 layers where Euclidean does connect (i.e. excluding 17/20/31), stable r^* from the two metrics: **agrees exactly in 10 layers, Euclidean reports a higher r^* in 13, cosine reports higher in 6**. Layer 15's original finding — that Euclidean and cosine can genuinely disagree on intrinsic dimension away from any disconnection issue ($r^* \approx 6$ vs. $r^* \approx 2$ there) — generalizes: Euclidean-higher is the single most common pattern when they disagree, but it is not universal, and a meaningful minority of layers (6) show the opposite. This reinforces the original caveat: switching to cosine everywhere is a real modeling tradeoff, not a free correctness fix — it discards magnitude information that appears to carry real structure at roughly 40% of layers, concentrated on (but not limited to) the disagreement-favors-Euclidean direction.

A real bug found and fixed while running the full sweep: at layer 25, ARPACK's iterative eigensolver (`scipy.sparse.linalg.eigs` , used by `diffusion_eigenvectors` for efficiency on the full eigendecomposition) failed to converge at one of the swept bandwidths and raised an uncaught `ArpackNoConvergence` , crashing the run 25/32 layers in. This makes sense in hindsight: the sweep deliberately probes extreme bandwidths, and at some of them the kernel matrix's eigenspectrum becomes tightly clustered/near-degenerate — exactly the condition that makes an iterative Lanczos-type solver struggle. **Fix:** `diffusion_eigenvectors` now catches `ArpackNoConvergence` and falls back to a dense solver (`scipy.linalg.eig`), the same fallback already used for matrices too small for ARPACK's constraints — cheap here regardless of which reason triggers it, since these matrices are only landmark-sized (at most a few hundred rows). Confirmed with a regression test that mocks the failure directly (ARPACK's non-convergence isn't reliably reproducible on demand) and verifies the fallback returns a correct result.

Caveat: the multiscale sweep uses wikitext throughout (200 sequences), not the kill switch's wikipedia/1000-sequence sample, for the CDN-reliability reason noted in §5 — see `methodology.md` §2 for why these aren't a byte-for-byte comparable pair.

3. A Second, Open Question: Does Low r^* Justify Narrower Experts?

Separately from the disconnection issue, a conceptual gap was identified in the project's own reasoning: **r^* and the spectral gap measure the dimensionality of the routing manifold — how many coarse directions organize which expert a token goes to. They say nothing about the dimensionality of what each expert's FFN needs to compute once a token arrives.**

The diffusion coordinates are an explicit low-pass filter (each mode is downweighted by λ^t). That's appropriate for robust routing. But the project's own framing of the FFN's job (attention blurs context together; the FFN sharpens it back to precision) is specifically about resolving high-frequency detail — exactly what the routing coordinates are designed to discard. `expert_ffn.py`'s width formula ($d_{ff}(k) \approx d_{ff} / K$) and the G2 success criterion ($\geq 40\%$ FFN FLOP reduction) both assume, as an *additional, unverified* leap beyond r^* , that tokens sharing a coarse cluster also share which FFN "pattern detector" neurons they need.

A diagnostic to test this directly was built (`geometry/ffn_specialization.py` , `scripts/ffn_specialization.py`): using the pretrained dense model's real FFN, it clusters tokens by diffusion coordinates (standing in for router assignment) and measures, per cluster, how much of that cluster's real neuron-activation mass a width- k slice of its own top neurons captures versus a cluster-

agnostic globally-shared slice of the same size. A large gap supports narrow per-cluster experts; a gap near zero would falsify the width-reduction hypothesis independent of whatever r^* says.

3.1 First run: invalidated by a clustering artifact

The first real-model run clustered tokens using plain k-means on **raw**-Euclidean diffusion coordinates. This collapsed at every layer tested, including the clean control (layer 15): one cluster absorbed ~99% of all pooled tokens, with the remaining "clusters" as 2–32-token singletons — the same extreme-norm tokens driving the disconnection issue (§2) get isolated by k-means regardless of K, leaving everything else lumped into one undifferentiated blob. Not a meaningful semantic partition, so the resulting gain/overlap numbers from that run are not trustworthy and are superseded below. (Root cause: the project's real

`DiffusionRouter` avoids exactly this collapse with a load-balancing loss — `coefficient_of_variation_loss` — that this diagnostic's plain k-means didn't replicate.)

Fix: cluster on cosine-normalized diffusion coordinates instead (consistent with §2.4/§4's finding that cosine is more robust to outlier-norm tokens), while keeping the actual FFN activations being measured raw and unnormalized. Verified this restores balanced clustering: layers 1 and 15 now split roughly 45–51% max-cluster-share at K=4 (a real 4-way partition); layer 31 improved from 99% to 83% (still the most skewed of those checked, plausibly reflecting genuine — not artifactual — concentration at that layer, given it also has $r^*=1$ there).

3.2 Second run: weak specialization signal across the board

Layer	K=4 gain / jaccard	K=8 gain / jaccard	K=16 gain / jaccard	Verdict pattern
1	0.013 / 0.26	0.064 / 0.16	0.075 / 0.12	does-not-support → ambiguous
2	0.081 / 0.28	0.103 / 0.17	0.089 / 0.12	ambiguous throughout
4	0.085 / 0.26	0.084 / 0.21	0.069 / 0.17	ambiguous throughout
15 (control)	0.020 / 0.47	0.020 / 0.37	0.021 / 0.30	does NOT support, at every K
31	0.026 / 0.75	0.031 / 0.59	0.034 / 0.31	does NOT support, at every K

("gain" = mean specialization gain, own-cluster coverage minus shared-slice coverage; "jaccard" = mean pairwise top-neuron-set overlap between clusters. Verdict: supports if gain > 0.15 and jaccard < 0.5; does-not-support if gain < 0.05; else ambiguous.)

Layer 15 is the most trustworthy data point here — the best-balanced clustering, and a clean, consistent negative signal across all three K values: specialization gain stays near-zero and neuron-set overlap between clusters stays moderate-to-high. Layers 1, 2, and 4 show a somewhat more positive but still modest signal (never crossing into a confident "supports" verdict), and layer 31 — despite still-imperfect clustering balance — shows the weakest signal of all, with jaccard overlap up to 0.75 at K=4.

Reading across all five layers: there is currently little to no compelling evidence that diffusion-coordinate routing clusters correspond to genuinely specialized FFN neuron usage. This is a materially different, more cautious conclusion than the first (artifact-driven) run suggested, and it's a real risk to the G2 success criterion (≥40% FFN FLOP reduction) independent of whatever r^* says about routing feasibility. This diagnostic only covered 5 of 32 layers on a modest 200-sequence sample — not yet conclusive enough

to treat as final, but concerning enough that G2 should not be assumed to hold without further, wider testing.

This reading has an unresolved ambiguity, though: the dense model was never trained with any incentive to organize around diffusion-cluster boundaries, so a weak result against diffusion clustering specifically can't distinguish "not achievable" from "achievable, just not found by this untrained, fixed routing signal" — a trained MoE's load-balancing loss actively reshapes specialization in a way nothing training-free can simulate. §3.3 resolves that ambiguity.

3.3 Third run: an oracle ceiling separates "not achievable" from "not found" — first on 5 layers, then all 32

To resolve the ambiguity above without training anything, each layer was ALSO clustered directly by its own FFN activation pattern (PCA-reduced to the same 32 dimensions the diffusion router uses) — the best-case K-way partition for this exact metric, establishing a training-independent ceiling on achievable specialization. Comparing that oracle's specialization gain against diffusion clustering's, and measuring their agreement (Adjusted Rand Index), separates two previously-conflated questions: is specialization achievable at all, and if so, does the current router find it?

First pass, 5 layers:

Layer	Oracle ceiling (gain, K=4/8/16)	Diffusion gain	ARI (diffusion vs. oracle)	Verdict
1	0.005 / 0.006 / 0.005	0.013 / 0.064 / 0.075	0.04 / 0.03 / 0.01	Not separable by any partition
2	0.116 / 0.153 / 0.110	0.081 / 0.103 / 0.089	0.13 / 0.05 / 0.05	Achievable — diffusion routing misses it
4	0.052 / 0.110 / 0.089	0.085 / 0.084 / 0.069	0.06 / 0.06 / 0.02	Achievable (weaker) — mostly missed
15 (control)	0.045 / 0.046 / 0.043	0.020 / 0.020 / 0.021	0.04 / 0.05 / 0.06	Not separable by any partition (borderline)
31	0.046 / 0.043 / 0.045	0.026 / 0.031 / 0.034	0.28 / 0.15 / 0.11	Not separable — ceiling too low to matter despite higher agreement

At 5 layers, this read as a roughly even split: layers 1/15/31 showed real structural evidence against width reduction; layers 2/4 showed a genuine ceiling the router wasn't finding.

Full 32-layer pass (`results/ffn_full32/` , same methodology, K=8 shown — full K=4/8/16 data in the saved JSON) tells a more lopsided story:

Bucket (at K=8)	Count	Layers
Not separable by any partition (oracle gain < 0.05)	6	1, 20, 21, 23, 25, 28
Achievable, but diffusion routing largely misses it (ARI < 0.2)	23	0, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 22, 24, 26, 27, 31

Achievable, and partially recovered (ARI $\geq$ 0.2)	3	10, 29, 30
--	---	------------

Layer 3 — untested in the 5-layer pass — has the single highest oracle ceiling of all 32 layers (gain 0.142 at K=8), edging out layer 2 (0.126), the earlier standout. Layer 30 pairs a strong ceiling (0.133) with the best router agreement found anywhere (ARI 0.255) — the closest thing in this investigation to "the geometry has real structure and the current router is actually finding some of it." Layer 15 (the original control) now reads as achievable-but-missed (oracle 0.066, ARI 0.056) rather than not-separable — a reminder that these figures move with the sample (wikitext vs. the mixed sampling used earlier; see `methodology.md` §2's caveat on cross-run comparability) and the 5-layer numbers shouldn't be read as precise, just directionally consistent, which they are for layers 1, 2, 4, and 31.

The headline shift from 5 layers to 32: width reduction looks *more*, not less, achievable at the geometry level than the initial sample suggested — 23 of 32 layers have real headroom, not just 2. But the router recovering that headroom is the exception (3 of 32), not the rule. This sharpens, rather than resolves, the case made in §3.4 below: the bottleneck is overwhelmingly the *routing signal*, not the underlying FFN structure. It also changes the shape of the selective-architecture question — with this much headroom this broadly distributed, `layers_to_replace` may end up excluding only a short list (the 6 not-separable layers) rather than being a short *inclusion* list built around 2–3 standout layers, contingent on the router instability found in §3.4 actually getting resolved first.

3.4 The bounded pilot fine-tune: two real bugs in the project's own routing code, found only by actually training something

Every diagnostic above is training-free, which left one question none of them could answer: would a *trained* router (with real gradients and a load-balancing loss) close the gap between diffusion's weak recovery and the oracle ceiling at layers 2/4? Testing this meant actually training something — freezing the entire pretrained Mistral-7B, splicing a trainable `PilotMoEBlock` in place of layer 2's dense MLP (reusing the project's own `NystromDiffusionMap`, `ExpertCentroids`, `DiffusionRouter`, `ExpertFFN` unmodified), and running a short, cheap fine-tune on a rented GPU. Doing this surfaced two real bugs in **pre-existing, shared project routing code** — bugs no training-free diagnostic in this investigation could have caught, since both are dynamics/scale issues that only manifest once centroids receive real gradients over many steps.

Bug 1 — unbounded centroid growth. The first full run showed `task_loss` falling substantially (closing roughly 78% of the gap to a dense baseline computed on the same held-out batch), which looked encouraging — but `load_loss` read exactly `0.0000` at all 300 steps while `sep_loss` grew from -3 to -19,185, a roughly 4-order-of-magnitude blowup. Root cause: `centroid_separation_loss`'s gradient has no upper bound on how far it pushes centroids apart (confirmed intentional by the existing `test_more_spread_centroids_give_more_negative_loss`), so nothing stops centroids from drifting arbitrarily far outside the actual data's coordinate range — at which point every token becomes roughly equidistant from every centroid in relative terms, collapsing the router toward uniform dispatch. **Fix:** `ExpertCentroids.clip_norm()`, a new method that caps each centroid's norm at a multiple of `landmark_scale` (the typical landmark-to-landmark spacing) after every optimizer step — confirmed by a reproduction test (`test_clip_norm_prevents_the_runaway_growth_a_real_pilot_run_hit`) and by re-running: `sep_loss` now plateaus (roughly -0.01 to -30, occasionally low hundreds) instead of diverging.

Bug 2 — deeper, and universal: the router wasn't routing at all. Re-running with bug 1 fixed, `load_loss` was *still* exactly `0.0000` at every step. Checked directly against real Mistral-7B layer-2 activations (not just the pilot): real diffusion coordinates measured $|\Psi_t| \approx 6 \times 10^{-7}$ (top eigenvalues ~ 0.02 – 0.05 , and $\Psi_t = \text{eigenvector} \cdot \text{eigenvalue}^t$ with `diffusion_t=3` shrinks this further), and the router's tempered

(`tau=0.1`) dispatch softmax read **exactly** `[0.125, 0.125, ..., 0.125]` for every expert, on every real token tested — not close to uniform, *identically* uniform to floating-point precision. `tau=0.1` (inherited from `configs/base_config.yaml`) implicitly assumes router logits are $O(1)$ scale; real diffusion coordinates are many orders of magnitude smaller, so no signal survives the softmax regardless of which centroid is actually closest. This is not a pilot-specific issue — `DiffusionMoELayer` (the production layer) shares the exact same `DiffusionRouter` / `NystromDiffusionMap` combination and would hit the identical degeneracy the moment it was ever trained.

Fix: both `PilotMoEBlock` and `DiffusionMoELayer` now route on `$\Psi_t$` and centroids rescaled by `landmark_scale` before the tempered softmax, making `tau` operate in a consistent, dimensionless unit ("multiples of typical landmark spacing") instead of each layer's own — and, empirically, unpredictable — raw coordinate magnitude. Fixing this exposed a **second-order bug in the fix's own dependency**: `landmark_scale` 's zero-guard epsilon (`+1e-8`) was itself large enough to dominate and silently cap the rescaling when the true scale was smaller still (down to $\sim 1e-16$ in one stress test) — tightened to `1e-30` , confirmed by `test_scale_normalization_prevents_uniform_dispatch_from_tiny_diffusion_coordinates` .

Corrected full 300-step run — a more honest, and more modest, result:

	task_loss
Dense baseline	2.524
Step 0 (random init)	5.910
Best point (step 270)	3.810
Final (step 299)	4.476

Gap closed: roughly 42% by the end (roughly 62% at the best point reached, step 270) — real learning, but substantially less than the *buggy* runs showed (78%). That's expected, not a regression: the earlier number measured how well an ensemble of 8 FFNs under near-uniform (degenerate) dispatch could jointly approximate the dense computation — an easier problem than genuine specialized routing, which is what's actually being tested now.

A new, real pathology surfaced now that routing works at all: training is unstable. `load_loss` oscillates across the run, hitting exactly `7.0000` at several steps (10, 140, 180, 220) — for `n_experts=8` , `coefficient_of_variation_loss` 's documented maximum, meaning the router **fully collapsed onto a single expert** at those points — then recovering toward balance elsewhere (as low as ~ 0.0001). Plausible cause: `mu=0.01` (the default load-balance weight, inherited from `configs/base_config.yaml`) may be too weak relative to the task-loss gradient at this learning rate over only 300 steps, letting the router swing between collapse and balance rather than settling.

Reading: real, diffusion-based specialization is learnable here — the architecture isn't broken, and this is a materially different (and more trustworthy) result than either bugged run produced — but the current default hyperparameters don't yet give *stable* training. That instability is itself a legitimate, separate finding from the routing bugs, and worth tuning (stronger `mu` , a router warmup phase, more steps) before drawing further conclusions from this specific configuration.

Two follow-up tuning attempts, and why they were inconclusive:

Run	<code>mu</code>	<code>centroid_refresh_steps</code>	Final task_loss	Gap closed	<code>load_loss</code> pattern

Corrected (above)	0.01 (default)	20 (default)	4.476	roughly 42% (roughly 62% best)	Full collapses (7.0) at steps 10, 140, 180, 220 — 5/13 near-collapse events land on refresh-interval multiples
Refresh- fix test	0.05	10,000 (effectively fit- once)	4.467	roughly 42%	sep_loss fully stabilized (confirms refresh- schedule was contributing), but load_loss got worse: sustained collapse (6.4–6.9) in the last third of training
Stronger μ	0.3	10,000	5.048	roughly 25–46% (worst final number of the three)	Still oscillates (6.3– 6.7 spikes at steps 140–260), recovers by the end rather than staying collapsed — a different failure shape, not a clear improvement

The second run isolated and confirmed a real contributor (periodic re-fitting of the Nyström landmarks was rotating the diffusion-coordinate frame under the router mid-training — holding landmarks fixed after the first fit removed that source of instability, visible in `sep_loss` 's clean plateau). But raising `mu` 6× on top of that fix did not reduce collapse frequency or severity, and produced the worst final `task_loss` of the three runs — evidence against "just weight load-balancing more heavily" as the fix. Notably, the `mu=0.05` and `mu=0.3` runs were bit-identical through step 80 despite the 6× weight difference, indicating the load-balance gradient's magnitude is small enough that even a large reweighting takes many steps to visibly diverge.

Assessment: with only 300 steps and `batch_size=4` (~1,024 tokens/step), the load-balance signal itself is estimated from a small, noisy sample — collapse/recovery cycling this size of run may partly reflect estimation noise rather than a single tunable cause. Further blind hyperparameter search under this noise floor was judged unlikely to be conclusive; the more promising next lever (untried) is increasing effective batch size (larger batch or gradient accumulation) to get a stabler per-step load estimate, which is a standard consideration for load-balancing losses generally rather than a project-specific guess. This is left as an open item (§7) rather than pursued further in this pass, in favor of returning to the three items that were on hold going into the pilot detour.

Why this matters beyond the pilot: both bugs live in shared code (`routing/centroids.py` , `routing/separation.py` , `routing/router.py`) used by the production `DiffusionMoELayer` , not anything pilot-specific. Had Phase 2 training gone ahead on the strength of the training-free diagnostics

alone, it would have hit both of these — likely silently, since a collapsed/uniform router doesn't necessarily crash, it just quietly fails to do the one thing (diffusion-based specialization) the whole project is about.

3.5 Resolving the instability: larger batch size, a seed-dependence diagnosis, and noisy top-k gating

§3.4's tuning attempts left the untried lever as increasing effective batch size, to get a less noisy per-step load estimate. Following up on that, plus a further, more specific diagnosis, resolved most of the open instability.

Batch size 4→16 (all else held at the original defaults: `mu=0.01` , `centroid_refresh_steps=20`) produced a real, large improvement: `task_loss` closed roughly 86% of the gap to baseline (vs. roughly 42% at `batch_size=4`), and `load_loss` never once hit the full-collapse ceiling across 300 steps (vs. multiple exact hits before). Mean `load_loss` was still well above zero (roughly 1.43), so imbalance wasn't eliminated — but the worst failure mode (total collapse) was gone.

A new diagnostic — logging which specific expert wins the dispatch each step, not just the aggregate `load_loss` scalar — found something batch size alone didn't explain: one pair of experts (indices 1 and 7) won 83% of all 300 steps' dispatch. Rerunning the identical configuration with a different random seed reproduced comparably severe concentration, but onto a **completely different** set of experts (3, 4, 5) — ruling out a real 2-mode structure in layer 2's geometry and confirming the cause is k-means++ centroid-initialization luck: whichever centroids happen to land closest to the bulk of the token distribution at seed time win an early lead in the router's tempered softmax, which the router's own training dynamics then reinforce rather than correct (a "rich-get-richer" pattern well documented for learned MoE routers generally).

Fix attempted: noisy top-k gating (Shazeer et al., 2017) — Gaussian noise added to router logits during training only, to give under-favored experts a periodic chance to win regardless of centroid-initialization luck. Added to the shared `DiffusionRouter` (off by default, `noise_std=0.0` , so no change to any existing behavior). The first real-run attempt (`noise_std=1.0`) **made things worse, not better** — a genuine miscalibration, not a failure of the technique: noise is added before the router's `tau=0.1` temperature division, so raw `noise_std=1.0` becomes an effective ~10 in the actual softmax input, large enough to dominate the real routing signal entirely. Result: full collapses jumped from 0 to 51 of 300 steps, and the same dominant expert's share rose from 83% to 95% — noise had made per-step outcomes more erratic without actually broadening which experts got used on average, and the training dynamics ended up reinforcing the pre-existing favorite even more strongly.

Corrected run, `noise_std=0.1` (roughly 10× smaller, chosen to land noise back in the same effective-magnitude range validated in a controlled unit test): the fix worked as intended.

	Clean (no noise)	Noisy, std=1.0 (miscalibrated)	Noisy, std=0.1 (corrected)
Gap closed (final / best)	86% / 96%	60% / 104%	82% / 93%
Full-collapse steps	0 / 300	51 / 300	0 / 300
Mean <code>load_loss</code>	1.43	4.18	1.32
Experts with meaningful share	2 (83% combined)	1 (95%)	5 (top one only 46%)

At the corrected magnitude, noisy gating matched or slightly beat the clean run's collapse-avoidance and mean load balance, while meaningfully broadening which experts actually receive tokens (5 experts with non-trivial share instead of 2), at the cost of a small, plausibly noise-level dip in task-loss gap closure. This is a genuine, working mitigation for the router-collapse pathology — not a complete fix (still a single-seed, single-layer result, and task-loss gap closure is a bit below the best clean run), but a real step past where §3.4 left off.

A recurring infrastructure lesson worth recording alongside the research result: getting each of these pilot runs to actually execute required fixing a real, repeat-offending problem — HuggingFace Hub's streaming dataset reader stalling indefinitely on a flaky CDN path (this happened on two different datasets across this investigation, most recently traced to HF's newer "Xet" CDN backend issuing many small, separately-connected byte-range requests). Pre-fetching files with `hf_hub_download` does not fix this on its own, since Hub *streaming* reads go through a different code path that re-fetches over the network regardless of local cache contents — the fix that actually worked was a new `local_data_files` option on `StreamingTextDataset`, pointing it directly at already-downloaded local parquet files, bypassing network reads for training data entirely. See `methodology.md` §4 for detail.

3.6 Verifying the fixes actually hold in the production training path — and finding three more bugs

Every fix and result up to this point was exercised only through the pilot's own splice-into-a-frozen-pretrained-model setup, never through the project's actual production path (`scripts/train.py` → `Trainer` → `DiffusionMoETransformer`, training the from-scratch architecture directly).

Recommendation 4 flagged this as an open gap. Closing it meant running a short but genuinely real training step through that exact path — small model (`d_model=128`, 2 layers, both replaced with `DiffusionMoELayer`), real streamed wikitext data, real bf16 mixed-precision (the project's own default), a handful of real optimizer steps.

It didn't just confirm the two known fixes — it crashed outright, three more times, for a reason nothing before this had reason to catch: NumPy has no `bf16` dtype at all, and `torch.cdist` has no `bf16` implementation either. Both are hard crashes, not precision concerns. `DiffusionMoELayer._compute_diffusion_coords`, `ExpertCentroids.initialise_from_batch`, and `landmark_scale / centroid_separation_loss` (via `torch.cdist`) each converted a real bf16 tensor without first casting to float32 — a step the pilot's `PilotMoEBlock` happened to already do (via its own `DtypeCastWrapper`, added for an unrelated reason), which is exactly why nothing before this surfaced it: the pilot's training-based validation used a code path that incidentally avoided the bug the production path actually has. **This is precisely the scenario recommendation 4 was written to guard against** — a fix confirmed working under one training setup, silently broken under the one that actually matters.

Fixed (all three call sites now cast to float32 before the numpy/cdist operation; gradients flow through the upcast unaffected): `models/moe_layer.py`, `routing/centroids.py`, `routing/separation.py`. Confirmed with a direct bf16 forward+backward regression test, and by rerunning the real smoke test end to end — 8 real steps completed cleanly, `load_loss` and `sep_loss` both finite and non-degenerate throughout (no exact- `0.0` uniform dispatch, no runaway growth), the same two original bugs' fixes holding correctly in the path that actually matters for Phase 2.

Separately, wiring `ExpertCentroids.clip_norm()` (bug 1's fix, §3.4) into the production `Trainer` itself was also found missing during this work: the method existed and was proven correct (`test_clip_norm_prevents_the_runaway_growth_a_real_pilot_run_hit`), but the *only* caller anywhere in the codebase was the pilot script's own bespoke training loop — `Trainer.train_step()` never called it, so real training through the production path had no protection against the exact unbounded-centroid-growth failure mode bug 1 documented. Fixed by calling it from `Trainer.train_step()` directly,

after every optimizer step, for every `DiffusionMoELayer` block present — so the safeguard now applies automatically to any training run through this path, not just ones that remember to ask for it.

4. Attention-Sink Follow-Up: Which Tokens, Actually?

The disconnection diagnosis above (§2.3–2.4) established *that* outlier-norm tokens fragment the kernel graph and traced it to the general "attention-sink" mechanism documented for Llama/Mistral-family models. It didn't establish *which* tokens are actually responsible. A token-identity diagnostic (`geometry/sink_diagnostics.py` , `scripts/sink_token_diagnostic.py`) was built to answer that directly against the real model, and the answer revised the initial story in an important way.

4.1 Not position — token identity

The classic "attention sink" literature (Xiao et al., 2023) frames the phenomenon as anchored to the first token(s) of the sequence (often the BOS token). That predicts outliers clustering at position 0. **The real data shows the opposite: `frac_at_position_0 = 0.00` at every single flagged layer.** Instead, the outliers are overwhelmingly one specific token — the newline character (`\n` , token id 13) — wherever it happens to occur in the sequence:

Layer	Dominant outlier token	Occurrences (of 41 flagged)	Mean norm of that token
1	<code>\n</code>	27	0.63 (vs. layer median 0.15)
2	<code>\n</code>	27	0.69 (vs. layer median 0.18)
4	<code>\n</code>	27	0.80 (vs. layer median 0.30)
31	<code>\n</code>	27	176.10 (vs. layer median 8.73)

This is a documented, related-but-distinct phenomenon to the textbook BOS-sink story: some models spread their "no-op" attention mass across frequent, low-semantic-content delimiter tokens (newlines, punctuation) rather than concentrating it solely on the first token. Mechanistically it's the same pressure (softmax needs a cheap escape valve for queries with nothing relevant to attend to) — this model just realizes it through a different, more distributed carrier.

Critically, the same token is responsible at **every** flagged layer, from 1 all the way to 31, just growing in magnitude with depth (0.63 → 0.69 → 0.80 → 176). That's a clean signature of one mechanism compounding through the residual stream — not a distinct process appearing fresh near the output, which argues against a "selection sharpening near prediction time" story and for the "compounding structural artifact" story, just via a different concrete token than originally assumed.

(The "66% of outliers at the last valid position" statistic from the initial position-based pass is very likely a truncation artifact — wikitext's frequent short lines/paragraph breaks interacting with a fixed 256-token cutoff — not evidence of an "importance near prediction" effect. Not treated as a real finding.)

4.2 Is it newline specifically, or delimiters/punctuation as a class?

Extending the diagnostic to aggregate norm by token identity (not just the extreme top-1% cut, which is biased toward whichever token simply occurs most often) gives a more nuanced answer — **the class-level effect only shows up at layer 31, not at 1/2/4:**

Layer	Delimiter-class mean norm	Content-class mean norm	Ratio
-------	---------------------------	-------------------------	-------

1	0.172	0.158	1.1× — negligible
2	0.215	0.186	1.2× — negligible
4	0.330	0.307	1.1× — negligible
31	18.4	8.5	2.2× — a real group effect

At layers 1/2/4, once newline itself is set aside, the rest of each layer's top-ranked tokens are ordinary content subwords ("itz", "Brook", "mother", "she", "June") — no punctuation pattern at all. At layer 31, a real class-level elevation appears: punctuation like \$, , and @ show meaningfully higher mean norm than content tokens.

Caveat before over-reading layer 31's list: several of the highest-ranked non-newline tokens there are ordinary content words tied to units and measurements — "km", "miles", "feet", "space" — alongside @ occurring unusually often (62 times). This is very likely picking up a wikitext-103 -specific formatting quirk: the corpus's raw text famously escapes numeric punctuation with @ (e.g. 5 @. @ 5 km , 1 @, @ 000), so units and @ co-occurring is probably a dataset artifact of that escaping convention, not a general property the model has learned about measurements. This would need checking against a different corpus before trusting it as a real semantic pattern.

4.3 Practical implication for mitigation

This changes which fix looks best. A blacklist-style mitigation ("exclude specific known sink tokens from geometry pooling") would almost fully neutralize layers 1, 2, and 4, where the pathology is essentially 100% attributable to one token. It's a much weaker fix for layer 31, where the elevation is more diffuse across many different, partly dataset-flavored token identities rather than one dominant culprit. That's a point in favor of the cosine-normalization approach (§2.4) as the primary architectural fix — it doesn't care about token identity at all, only raw magnitude — with token exclusion as a possible cheap, complementary layer for the specific early-layer newline case.

5. Incidental Bug Found: Broken wikitext Dataset Registry

While running the multiscale sweep, a background job stalled for 130+ minutes of CPU time retrying flaky `wikimedia/wikipedia` CDN shards with no progress. Switching to the registry's `wikitext` option surfaced a second, independent, pre-existing bug: `DATASET_REGISTRY["wikitext"]["path"]` pointed at the bare `"wikitext"` Hub repo id, which is deprecated and now redirects in a way the installed `datasets / huggingface_hub` version's URI parser doesn't follow, raising a hard error.

Fixed (`src/diffusion_moe/data/dataset.py`): repointed to `"Salesforce/wikitext"` , the dataset's current canonical location, and verified it streams correctly. This also affects `scripts/run_sweep.py` , which uses this exact registry entry for its Phase 2 wikitext-perplexity evaluation — it would have broken there too had it not been caught now.

6. Code Changes Summary

File	Change
<code>scripts/extract_geometry.py</code>	Fixed kill-switch threshold/message mismatch; added <code>likely_disconnected</code> , <code>pool_diagnostics</code> (eps, norm rat duplicate fraction)

src/diffusion_moe/geometry/nystrom.py	Added optional eps override to NystromDiffusionMap, controlled bandwidth sweeps
src/diffusion_moe/geometry/multiscale.py	New. Dyadic bandwidth sweep, cosine/L2 normalized stable-window detection
src/diffusion_moe/geometry/activation_capture.py	New. Shared model-hook/pooling logic, refactored extract_geometry.py for reuse
src/diffusion_moe/geometry/ffn_specialization.py	FFN neuron-usage specialization diagnostic (Geva et al style); added oracle_cluster_tokens_by_activation (token-independent specialization ceiling) and cluster_agreement (ARI/NMI) to separate "not achievable" from "not found by this router"
src/diffusion_moe/geometry/sink_diagnostics.py	New. Token-position and per-token-identity outlier diagnostics
scripts/multiscale_geometry.py	New. CLI to run the dyadic sweep against a real model metrics
scripts/ffn_specialization.py	New. CLI to test the width-reduction hypothesis against a real model; clustering fixed to use cosine-normalized coordinates after the first run's raw-Euclidean clusters were found to collapse into one dominant cluster; extended report oracle-ceiling vs. diffusion-recovery per layer
scripts/sink_token_diagnostic.py	New. CLI identifying which token positions/identities disconnect
src/diffusion_moe/data/dataset.py	Fixed broken wikitext registry entry
src/diffusion_moe/models/pilot_moe_block.py	New. PilotMoEBlock — trainable drop-in .mlp replacer for the bounded pilot, reusing NystromDiffusionMap/ExpertCentroids/DiffusionRouter/ unmodified
scripts/pilot_finetune.py	New. Freezes the pretrained model, splices in one PilotMoEBlock, trains only its params with a dense-batch comparison
src/diffusion_moe/routing/centroids.py	Added ExpertCentroids.clip_norm() — caps centroid norm to a multiple of landmark_scale after each optimizer step to fix the unbounded-growth bug found in §3.4
src/diffusion_moe/routing/separation.py	Extracted landmark_scale() (was inlined in centroid_separation_loss); epsilon tightened from 1e-30 after it was found to silently cap rescaling when the diffusion-coordinate scale is smaller than 1e-8
src/diffusion_moe/models/moe_layer.py	DiffusionMoELayer.forward now routes on landmark_scale-normalized coordinates — fixes the same router-deg bug in the production layer, not just the pilot

src/diffusion_moe/geometry/eigensolver.py	diffusion_eigenvectors now falls back to a dense eigendecomposition when ARPACK's iterative solver converge (§2.4), not just when the matrix is too small ARPACK's constraints
src/diffusion_moe/routing/router.py	Added noisy top-k gating (noise_std, off by default) to DiffusionRouter — see §3.5
src/diffusion_moe/data/dataset.py	Added local_data_files to StreamingTextDataset — by Hub streaming (a recurring flaky-CDN stall, hit on two different datasets across this investigation) by reading already-downloaded local parquet files directly — see and methodology.md §4
.gitignore	Fixed an unanchored data/ pattern that had silently excluded src/diffusion_moe/data/ (the real Python package) and tests/data/ from version control since the initial commit same bug class as an earlier rsync --exclude='data' same fix (anchor to /data/)
src/diffusion_moe/models/moe_layer.py	_compute_diffusion_coords now casts to float32 before .numpy() — NumPy has no bfloat16 dtype, so real bfloat16 training crashed outright without this (§3.6)
src/diffusion_moe/routing/centroids.py	initialise_from_batch — same .numpy()/bfloat16 fix as at
src/diffusion_moe/routing/separation.py	landmark_scale and centroid_separation_loss now cast float32 before torch.cdist, which has no bfloat16 implementation at all
src/diffusion_moe/training/trainer.py	Trainer.train_step now calls ExpertCentroids.clip_norm after every optimizer step, for every DiffusionMoELayer present — bug 1's fix (§3.4) existed but was previously never called by the pilot script's own training loop, never production Trainer

All changes are covered by tests validating both the plumbing and the specific claims being made (e.g. synthetic Swiss-roll and outlier-cluster scenarios for the multiscale module; disjoint-vs-uniform neuron-usage, cluster-balance, and oracle-ceiling-recovers-known-structure scenarios for the specialization module; start-anchored vs. end-anchored and delimiter-vs-content scenarios for the sink diagnostics; direct reproductions of both routing bugs — unbounded centroid growth and degenerate uniform dispatch from tiny diffusion coordinates — confirming each fix; a mocked ARPACK-non-convergence scenario confirming the dense fallback engages; a controlled synthetic scenario confirming noisy gating measurably reduces dispatch concentration; a real local parquet file confirming local_data_files reads correctly with no network access; a real bfloat16 forward+backward pass confirming the NumPy/cdist fixes; a deliberately-inflated centroid confirming Trainer.train_step actually calls clip_norm_). Full suite: 329 tests passing, clean lint, at last check.

7. Recommendations

1. **Do not greenlight Phase 2 training on the original run's result.** The disconnection artifact is now understood, but the underlying "clean" r^* (median 4–4.5, range 1–11, confirmed on the full sample)

is still surprisingly small relative to the guide's expectations, the full multiscale sweep confirms Euclidean/cosine disagreement is widespread (not just layer 15), and the full FFN-specialization result (§3.3) now shows the routing signal — not the geometry — is the dominant open risk.

2. **Done — the recommendation is a selective, not uniform, metric choice.** The full 32-layer multiscale sweep (§2.4) shows bandwidth choice alone resolves layers 1/2/4's disconnection but *cannot* resolve layers 17/20/31's — only cosine does, for those three specifically. Meanwhile, where both metrics connect, cosine is not a strict improvement: Euclidean reports a higher r^* than cosine at 13 of 29 comparable layers, cosine higher at only 6, suggesting real magnitude information is discarded at a meaningful fraction of layers if cosine became the uniform default. **Recommended:** keep raw Euclidean as the default routing metric, with a per-layer or disconnection-triggered fallback to cosine reserved for layers that need it (17, 20, 31, and any future model where the same pattern recurs) — not a project-wide switch.
3. **Done — full 32-layer oracle-ceiling coverage completed (§3.3).** 23 of 32 layers show a real, currently-unexploited specialization ceiling (far more than the 5-layer sample's 2); only 3 (10, 29, 30) show the current router recovering any meaningful part of it; only 6 layers are genuinely not separable by any partition. This substantially widens the case for width reduction at the geometry level while sharpening the case that the router, not the FFN structure, is the bottleneck — see recommendation 5.
4. **Done (§3.6), and it paid off a second time.** The bounded pilot fine-tune (§3.4) at layer 2 found two real bugs in shared routing code that no training-free diagnostic could have caught; both are fixed in the production `DiffusionMoELayer`. Running an actual short training step through the real production path (`Trainer` / `DiffusionMoETransformer`, not the pilot's splice) confirmed both fixes hold there — but also crashed outright on three more bugs the pilot's own setup had incidentally avoided (NumPy/ `torch.cdist` don't support `bfloat16` at all; real mixed-precision training hit this immediately), and found that bug 1's fix (`clip_norm`) was never actually wired into the production `Trainer`, only the pilot's own training loop. All five now fixed and confirmed with a clean 8-step real run. This is exactly the failure mode this recommendation existed to catch.
5. **Substantially resolved (§3.5).** A larger batch size (4→16) eliminated full router collapse on its own; logging per-step expert dominance (not just the aggregate loss) found the residual imbalance was k-means++ seed luck, confirmed by reproducing comparably severe concentration onto a *different* pair of experts under a different seed; noisy top-k gating, once correctly calibrated relative to `tau` (an initial attempt at 10× too strong made things measurably worse — a real miscalibration lesson, not a dead end), then matched the clean run's collapse-avoidance while meaningfully broadening expert utilization (5 experts with real share vs. 2). Not a complete fix — single-seed/single-layer evidence, and task-loss gap closure is still a bit below the best clean run — but a genuine, working mitigation, not just a diagnosis.
6. **Consider excluding a short list of known outlier tokens — chiefly the newline character — from geometry pooling** as a cheap, complementary mitigation for layers 1/2/4, where it's responsible for essentially the entire pathology. This is *not* a sufficient fix for layer 31 on its own (§4.3), where the effect is more diffuse across token identities.
7. **Done.** The full 32-layer single-scale kill switch reran cleanly with the fixed threshold and new diagnostics on the original 1000-sequence Wikipedia sample (§2.3) — reproduced the same 4 disconnected layers and r^* statistics as the earlier partial sample, confirming that result wasn't a sampling artifact.
8. **Verify the layer-31 punctuation/units pattern (§4.2) against a non-wikitext corpus** before treating it as a general model property rather than a `wikitext-103`-specific numeric-escaping artifact.